

# Library Management System Report

Gisma University of Applied Sciences

B100 Introduction to Computer Programming with Python

Assessment Title: Individual Project

Project Title: Library Management System

Student Name: Isaac Ankomah

Student ID: GH1056979

GitHub Repository: <https://github.com/isaacankomah/python-B100-assignment>

Date of Submission: 26 June 2026

## 1. Introduction

This project is a Python command-line Library Management System designed for a small community library. The purpose of the system is to help a librarian manage books, register members, search for books, and record borrowing and returning activities. In many small libraries, records may be kept manually in notebooks or spreadsheets. This can make it difficult to know which books are available, which members have borrowed books, and whether a borrowed book has been returned.

The system addresses this problem by providing a simple menu-driven program. Through the menu, the librarian can add books, register members, search for books, borrow books, return books, list all books, and list all registered members. The program also stores information in CSV files, so records are saved and can be loaded again when the program is restarted.

The scope of the project is limited to basic library operations. It does not include advanced features such as user login, overdue fine calculation, email reminders, or a graphical user interface. However, it demonstrates important Python programming concepts, including classes, objects, methods, loops, conditional statements, file handling, exception handling, and modular programming.

## 2. System Design

The system was designed using separate Python modules so that each part of the program has a clear responsibility. This makes the code easier to read, test, and maintain.

The `Book` class represents a single book in the library. It stores the book's ISBN, title, author, category, and availability status. Its methods allow the system to display book information, borrow a book, return a book, and update book details.

The `Member` class represents a registered library member. It stores the member ID, name, email address, and a list of borrowed books. The borrowed books are stored using ISBN numbers. The class includes methods for displaying member information, adding a borrowed book, returning a borrowed book, and listing borrowed books.

The `LoanRecord` class represents a borrowing transaction. It stores the loan record ID, member ID, ISBN, borrow date, return date, and status. This class helps the system track whether a book loan is active or has been returned.

The `Library` class connects the other classes together. It stores lists of books, members, and loan records. It includes methods for adding books, registering members, searching for books, borrowing books, returning books, and listing records.

The `data_handler.py` module handles CSV file operations. CSV files were used because they are simple and suitable for a beginner-level Python project. Python's CSV module supports reading and writing tabular data, including dictionary-based reading and writing through `DictReader` and `DictWriter` (Python Software Foundation, 2026a).

The overall structure of the system is:

```
Library
|-- manages Book objects
|-- manages Member objects
|-- manages LoanRecord objects
|-- saves and loads data using CSV files
```

The main files in the project are:

```
main.py
book.py
member.py
loan_record.py
library.py
data_handler.py
data/books.csv
data/members.csv
data/loans.csv
```

### 3. Implementation Overview

The program was implemented using object-oriented programming. Python classes were used to group related data and behaviour together. According to the Python documentation, classes allow data and functionality to be bundled together into objects (Python Software Foundation, 2026b). This idea was applied by creating separate classes for books, members, loan records, and the library.

Each class contains meaningful attributes and methods. For example, the `Book` class stores information about a book and includes methods such as `display_info()`, `borrow_book()`, `return_book()`, and `update_details()`. The `Member` class includes methods such as `display_info()`, `borrow()`, `return_book()`, and `list_borrowed_books()`. These methods make the code easier to understand because each method performs one clear task.

Loops and conditional statements are used in several parts of the system. In `main.py`, a `while True` loop keeps the menu running until the user chooses to exit. Conditional statements such as `if`, `elif`, and `else` are used to respond to the user's menu choice. Loops are also used in the `Library` class to search through books, members, and loan records.

File handling is implemented using CSV files. When the program starts, it loads existing books, members, and loan records from the CSV files. When the user adds a book, registers a member, borrows a book, or returns a book, the program saves the updated data. This means the system does not lose records after the program closes.

Exception handling is used in the menu system. If the user enters text instead of a number when selecting a menu option, the program catches the `ValueError` and displays a helpful message instead of crashing. This follows the purpose of exception handling, which is to manage errors that may occur during program execution (Python Software Foundation, 2026c).

The project is also modular. Instead of writing all the code in one file, the system is divided into separate files. This improves readability and makes testing easier. The code uses docstrings to explain the purpose of classes and functions. The coding style follows general PEP 8 principles such as readable names, consistent indentation, and clear organisation (Van Rossum, Warsaw

and Coghlan, 2001).

## 4. Testing and Demonstration

The system was tested in two main ways: individual class testing and full program testing.

First, the Book class was tested by creating a book object and displaying its information. The book was then borrowed and returned to check that the availability status changed correctly. The test data used in the project CSV file was:

```
ISBN: 9781234567890
Title: Introduction to Chemistry
Author: Isaac Ankomah
Category: Chemistry
```

Example output before borrowing:

```
ISBN: 9781234567890, Title: Introduction to Chemistry,
Author: Isaac Ankomah, Category: Chemistry, Status: Available
```

After borrowing the book, the status changed to:

```
Status: Borrowed
```

After returning the book, the status changed back to:

```
Status: Available
```

The Member class was tested by creating a member object and adding a borrowed book ISBN to the member's borrowed books list. The member record used in the CSV file was:

```
Member ID: ANKOLIB001
Name: Isaac Ankomah
Email: isaackkankomah@gmail.com
```

During borrowing, the member's borrowed books list showed the ISBN:

```
9781234567890
```

The LoanRecord class was tested by creating a loan record and checking whether it was active. After the book was returned, the status changed from active to returned. The saved loan record used record ID L001, member ID ANKOLIB001, ISBN 9781234567890, and status returned.

The full program was tested by running:

```
python main.py
```

The program displayed a menu with options such as:

```
Library Management System
1. Add book
2. Register member
4. Borrow book
5. Return book
8. Exit
```

A book was added using option 1, and a member was registered using option 2. The borrow option was then tested using member ID ANK0LIB001 and ISBN 9781234567890. The system displayed:

Book borrowed successfully.

The return option was also tested. After entering the same member ID and ISBN, the system displayed:

Book returned successfully.

Invalid input was tested by entering text instead of a number at the menu. The program handled the error and displayed:

Please enter a number from the menu.

This confirmed that the exception handling worked correctly. The CSV files were also checked after testing to confirm that book, member, and loan data were saved.

## 5. Reflection

One challenge in this project was understanding how the different classes should work together. At first, it was easier to think about books, members, and loan records separately. However, the `Library` class helped connect all these parts into one system. This showed the importance of planning class relationships before writing the full program.

Another challenge was saving and loading data from CSV files. The program needed to convert objects into dictionary rows before saving them. It also needed to rebuild objects from CSV rows when the program started. This helped improve my understanding of file input/output and data conversion in Python.

Through this project, I learned how object-oriented programming can make a program more organised. I also learned that testing small parts of a program before testing the whole system makes debugging easier. Testing the `Book`, `Member`, and `LoanRecord` classes separately helped identify problems before connecting everything in the main menu.

In the future, the system could be improved by adding librarian login, due dates for borrowed books, overdue fine calculation, and a graphical user interface. Another improvement would be to use a database instead of CSV files, especially if the library had many books and members.

Overall, the project demonstrates the main programming concepts covered in the module. It provides a useful basic system for managing books, members, and borrowing records in a small library.

## References

Python Software Foundation. 2026a. csv - CSV file reading and writing. [Online]. Available at: <https://docs.python.org/3.12/library/csv.html> [Accessed 26 June 2026].

Python Software Foundation. 2026b. Classes. [Online]. Available at: <https://docs.python.org/3.12/tutorial/classes.html> [Accessed 26 June 2026].

Python Software Foundation. 2026c. Errors and exceptions. [Online]. Available at: <https://docs.python.org/3.12/tutorial/errors.html> [Accessed 26 June 2026].

Van Rossum, G., Warsaw, B. and Coghlan, A. 2001. PEP 8 - Style guide for Python code. [Online]. Available at: <https://peps.python.org/pep-0008/> [Accessed 26 June 2026].